

Анализ факторов ДТП на основе проекта "Карта ДТП"

Описание проекта

В проекте анализируются данные некоммерческого сервиса «Карта ДТП», который собирает информацию о дорожно-транспортных происшествиях в России. Проект направлен на повышение безопасности дорожного движения и помогает выявлять особенности аварийности, оценивать проблемные участки и формировать рекомендации для улучшения дорожной инфраструктуры.

В исследовании используются данные о ДТП в Кировской и Московской областях, а также дополнительные таблицы с информацией об участниках происшествий и транспортных средствах. Данные содержат сведения о дате и времени ДТП, координатах, погодных условиях, состоянии дорожного покрытия, освещённости, количестве участников и пострадавших, а также о водительском стаже участников.

В рамках проекта проводится предварительная проверка данных, исследуется динамика ДТП по временным промежуткам и анализируется распределение происшествий среди водителей с разным стажем.

Цель проекта — изучить данные о дорожно-транспортных происшествиях и выявить особенности аварийности в Кировской и Московской областях.

Для достижения цели необходимо:

1. Провести первичное изучение данных.
2. Проверить данные на наличие пропусков и дубликатов.
3. Оценить корректность типов данных.
4. Определить, как менялось число ДТП по дням недели и месяцам.
5. Сравнить количество ДТП среди групп водителей с разным стажем.
6. Проанализировать различия в аварийности между регионами.
7. Подготовить выводы и рекомендации, которые могут быть полезны для повышения качества данных и безопасности дорожного движения.

Описание данных

Основные датасеты по Кировской и Московской областям содержат информацию о ДТП:
`Kirovskaya_oblast`, `Moscowskaya_oblast`

Дополнительные датасеты содержат сведения об участниках ДТП и сведения о транспортных средствах

```
In [7]: import pandas as pd
df=pd.read_csv('/datasets/Kirovskaya_oblast.csv')
```

Проверка ошибок в данных и их предобработка

```
In [8]: df['properties.participants_count'].value_counts()
```

```
Out[8]:
```

	count
properties.participants_count	
2	8611
3	2929
1	1533
4	909
5	353
6	109
7	44
8	9
9	9
10	5
12	2
11	2
30	1
14	1

dtype: int64

1) Изменение названий столбцов на оптимальные для работы

Выводим название столбцов для оценки качества

```
In [9]: df.columns
```

```
Out[9]: Index(['geometry.coordinates', 'id', 'properties.tags', 'properties.light',  
              'properties.point.lat', 'properties.point.long', 'properties.nearby',  
              'properties.region', 'properties.scheme', 'properties.address',  
              'properties.weather', 'properties.category', 'properties.datetime',  
              'properties.injured_count', 'properties.parent_region',  
              'properties.road_conditions', 'properties.participants_count',  
              'properties.participant_categories'],  
              dtype='object')
```

Части `geometry.`, `properties.` названий столбцов лишние и от них можно избавиться.

```
In [10]: df = df.rename(columns={'geometry.coordinates': 'coordinates',  
                                'properties.tags': 'tags',  
                                'properties.light': 'light',  
                                'properties.point.lat': 'point_lat'})
```

Подстрока `properties.` встречается многократно, и от неё сразу можно избавиться.

```
In [11]: df.columns = [c.replace("properties.", "") for c in df.columns]
```

2) Проверка пропусков в данных

Изучим пропуски в датасете `Kirovskaya_oblast.csv` .

```
In [12]: df.isna().sum().sort_values(ascending = False)
```

```
Out[12]:
```

	0
scheme	1137
address	674
point.long	32
point_lat	32
id	0
coordinates	0
tags	0
light	0
region	0
nearby	0
weather	0
category	0
datetime	0
injured_count	0
parent_region	0
road_conditions	0
participants_count	0
participant_categories	0

dtype: int64

Применение `isna().sum()` позволяет ответить на вопрос о пропусках в столбцах. Столбцы отсортированы по числу убывания пропусков и отфильтрованы ненулевые значения.

В столбце `Scheme` множество численных обозначений для схемы конкретной аварии. Заполнить его сложно, но в данный момент для анализа он не нужен.

```
In [13]: df['scheme'].value_counts()
```

```
Out[13]:
```

	count
scheme	
610.0	1051
300.0	1016
200.0	917
70.0	870
600.0	863
740.0	861
820.0	771

count	
scheme	
500.0	765
960.0	667
430.0	426
880.0	336
840.0	280
950.0	273
830.0	246
710.0	240
910.0	240
210.0	200
720.0	199
700.0	196
140.0	186
930.0	183
130.0	182
940.0	167
770.0	160
760.0	141
730.0	139
780.0	128
190.0	124
850.0	111
800.0	107
420.0	102
90.0	98
30.0	92
50.0	92
60.0	91
900.0	87
120.0	86
220.0	76
400.0	73
810.0	67
40.0	60
330.0	57
110.0	51
920.0	50
10.0	45

count	
scheme	
860.0	44
410.0	36
870.0	35
750.0	22
100.0	14
230.0	12
440.0	11
20.0	11
630.0	8
620.0	6
980.0	4
340.0	2
310.0	2
970.0	1

dtype: int64

В столбце address довольно много пропусков. При рассмотрении строк с пропусками аномалий не наблюдается, возможно, адрес просто не указан.

```
In [14]: df[df['address'].isna()]
```

	coordinates	id	tags	light	point_lat	point.long	nearby	r
18	[47.703667, 57.398549]	1981028	Дорожно-транспортные происшествия	В темное время суток, освещение отсутствует	57.398549	47.703667	[]	Яра
59	[47.969474, 57.105731]	2875566	Дорожно-транспортные происшествия	Светлое время суток	57.105731	47.969474	[]	Яра
63	[47.891099, 57.309193]	2875597	Дорожно-транспортные происшествия	Светлое время суток	57.309193	47.891099	['Многоквартирные жилые дома', 'Внутридворовая...	Яра
95	[47.906871, 57.020891]	2861063	Дорожно-транспортные происшествия	Светлое время суток	57.020891	47.906871	[]	Яра
130	[47.828239, 57.298419]	1984517	Дорожно-транспортные происшествия	Светлое время суток	57.298419	47.828239	['АЗС', 'Автостоянка (отделенная от проезжей ч...	Яра
...	...	...	...	...	...	...	...	...
14489	[51.079023, 56.227214]	1983961	Дорожно-транспортные происшествия	Светлое время суток	56.227214	51.079023	['Многоквартирные жилые дома', 'Внутридворовая...	Вс, Поля
14497	[51.293557, 56.251791]	1986098	Дорожно-транспортные	В темное время	56.251791	51.293557	['Многоквартирные жилые дома',	Вс, Поля

	coordinates	id	tags	light	point_lat	point.long	nearby	r
			происшествия	суток, освещение не включено			'Внутридворовая...	
14502	[51.072639, 56.221047]	1986546	Дорожно- транспортные происшествия	Светлое время суток	56.221047	51.072639	['Многоквартирные жилые дома', 'Внутридворовая...	Вс Поля
14511	[51.1186, 56.2417]	1980230	Дорожно- транспортные происшествия	В темное время суток, освещение отсутствует	56.241700	51.118600	[]	Вс Поля
14513	[50.8911, 56.1978]	1985930	Дорожно- транспортные происшествия	Сумерки	56.197800	50.891100	[]	Вс Поля

674 rows x 18 columns

Возникает вопрос, есть ли что-то необычное в авариях с пропусками адреса. Пропуск адреса может говорить о том, что рядом нет какого-либо объекта. В рамках исследования обратимся к столбцу `nearby` за помощью — в нём лежат ближайшие объекты.

```
In [15]: df[df['address'].isna()]['nearby'].value_counts()
```

	count
nearby	
[]	183
['Многоквартирные жилые дома', 'Внутридворовая территория']	161
['Жилые дома индивидуальной застройки']	41
['Многоквартирные жилые дома', 'Крупный торговый объект (являющийся объектом массового тяготения пешеходов и (или) транспорта)', 'Внутридворовая территория']	20
['Многоквартирные жилые дома']	14
...	...
['Производственное предприятие', 'Регулируемый ж/д переезд без дежурного']	1
['Остановка общественного транспорта', 'Регулируемый пешеходный переход', 'Автостоянка (не отделённая от проезжей части)']	1
['Мост']	1
['Многоквартирные жилые дома', 'Спортивные и развлекательные объекты']	1
['Многоквартирные жилые дома', 'Автостоянка (отделенная от проезжей части)', 'Одиночный торговый объект, являющийся местом притяжения транспорта и (или) пешеходов']	1

138 rows x 1 columns

dtype: int64

Не все значения пустые, а значит, гипотеза о том, что аварии с пропуском в `address` далеки от зданий, не подтверждается полностью. Похоже, этот столбец необязателен к заполнению.

Для минимизации пропусков в данных можно сделать некоторые поля обязательными к заполнению. Например, координаты и схему, если это необходимо для последующего анализа.

```
In [17]: df[df['point_lat'].isna()].head()
```

Out [17]:	coordinates	id	tags	light	point_lat	point.long	nearby	reg	
	391	[None, None]	1985738	Дорожно-транспортные происшествия	Сумерки	NaN	NaN	[]	Шабалинский рай
	1029	[None, None]	1986653	Дорожно-транспортные происшествия	Светлое время суток	NaN	NaN	[]	Свечинский рай
	2198	[None, None]	1986337	Дорожно-транспортные происшествия	В темное время суток, освещение отсутствует	NaN	NaN	['Жилые дома индивидуальной застройки']	Нолинский рай
	2227	[None, None]	1985083	Дорожно-транспортные происшествия	В темное время суток, освещение отсутствует	NaN	NaN	[]	Нолинский рай
	2269	[None, None]	1985078	Дорожно-транспортные происшествия	Светлое время суток	NaN	NaN	['Жилые дома индивидуальной застройки']	Нолинский рай

3) Наличие явных и неявных дубликатов в данных

Выясним наличие явные дубликатов в данных и их процент от общего объема данных

```
In [18]: df.duplicated().sum()
```

```
Out[18]: np.int64(0)
```

Проверим, являются ли уникальные идентификаторы уникальными.

```
In [20]: df['id'].nunique() == df['id'].count()
```

```
Out[20]: np.True_
```

Число уникальных идентификаторов равно общему числу идентификаторов, что говорит о том, что каждый из них — уникален.

4) Проверка корректности типов данных

```
In [21]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14517 entries, 0 to 14516
Data columns (total 18 columns):
 #   Column                                Non-Null Count  Dtype

```

```

0    coordinates      14517 non-null object
1    id               14517 non-null int64
2    tags             14517 non-null object
3    light            14517 non-null object
4    point_lat        14485 non-null float64
5    point.long       14485 non-null float64
6    nearby          14517 non-null object
7    region           14517 non-null object
8    scheme           13380 non-null float64
9    address          13843 non-null object
10   weather          14517 non-null object
11   category         14517 non-null object
12   datetime         14517 non-null object
13   injured_count    14517 non-null int64
14   parent_region    14517 non-null object
15   road_conditions  14517 non-null object
16   participants_count 14517 non-null int64
17   participant_categories 14517 non-null object
dtypes: float64(3), int64(3), object(12)
memory usage: 2.0+ MB

```

```
In [22]: df.columns
```

```
Out[22]: Index(['coordinates', 'id', 'tags', 'light', 'point_lat', 'point.long',
               'nearby', 'region', 'scheme', 'address', 'weather', 'category',
               'datetime', 'injured_count', 'parent_region', 'road_conditions',
               'participants_count', 'participant_categories'],
              dtype='object')
```

```
In [23]: df.head()
```

```
Out[23]:
```

	coordinates	id	tags	light	point_lat	point.long	nearby	region	s
0	[47.875603, 57.24379]	1983180	Дорожно-транспортные происшествия	Светлое время суток	57.244	47.875603	[]	Яранский район	
1	[47.87903, 57.304807]	2889433	Дорожно-транспортные происшествия	Светлое время суток	57.305	47.879030	['Административные здания', 'Нерегулируемый пе...]	Яранский район	
2	[47.840781, 57.297156]	2591208	Дорожно-транспортные происшествия	Сумерки	57.297	47.840781	['Жилые дома индивидуальной застройки', 'Нерег...]	Яранский район	
3	[47.834365, 57.244775]	2577639	Дорожно-транспортные происшествия	Светлое время суток	57.245	47.834365	['Жилые дома индивидуальной застройки']	Яранский район	
4	[47.968197, 57.357738]	1981026	Дорожно-транспортные происшествия	Светлое время суток	57.358	47.968197	['Нерегулируемый перекрёсток неравнозначных ул...]	Яранский район	

- В `coordinates` располагается пара значений. С ней неудобно взаимодействовать, к тому же отдельно существуют столбцы `point_lat` , `point_long` , которые более самостоятельны.
- Для столбца `id` достаточно `int32` , что сэкономит место.

- Для `participants_count` , `injured_count` достаточно `int8` .
- Для `scheme` — `int16` . Нужно заменить пропуски заглушкой и округлить значения.
- Есть ряд столбцов (`nearby` , `weather` , `road_conditions` , `participant_categories`), в которых есть списки. Для решения поставленных задач преобразование не требуется. Оставляем тип `object` .
- Строковые столбцы (`tags` , `light` , `region` , `address` , `category` , `parent_region`) можно оставить в текущем виде, потому что в них действительно располагаются строки.

In [24]: `df.head()`

Out [24]:

	coordinates	id	tags	light	point_lat	point.long	nearby	region	s
0	[47.875603, 57.24379]	1983180	Дорожно-транспортные происшествия	Светлое время суток	57.244	47.875603	[]	Яранский район	
1	[47.87903, 57.304807]	2889433	Дорожно-транспортные происшествия	Светлое время суток	57.305	47.879030	['Административные здания', 'Нерегулируемый пе...]	Яранский район	
2	[47.840781, 57.297156]	2591208	Дорожно-транспортные происшествия	Сумерки	57.297	47.840781	['Жилые дома индивидуальной застройки', 'Нерег...]	Яранский район	
3	[47.834365, 57.244775]	2577639	Дорожно-транспортные происшествия	Светлое время суток	57.245	47.834365	['Жилые дома индивидуальной застройки']	Яранский район	
4	[47.968197, 57.357738]	1981026	Дорожно-транспортные происшествия	Светлое время суток	57.358	47.968197	['Нерегулируемый перекрёсток неравнозначных ул...]	Яранский район	

Исследовательский анализ

Какое число ДТП случилось в каждый день недели. Разбивка числа происшествий по месяцам

Импортируем библиотеку и добавляем необходимые столбцы.

In [3]: `import datetime as dt`

In [25]: `df['datetime'] = pd.to_datetime(df['datetime'])`

In [26]: `df['day_of_week'] = df['datetime'].dt.day_name()`

In [27]: `df['month'] = df['datetime'].dt.month`

Выводим число ДТП по дням недели и месяцам.

```
In [28]: df['day_of_week'].value_counts()
```

```
Out[28]:
```

	count
day_of_week	
Friday	2344
Saturday	2246
Sunday	2054
Monday	2010
Tuesday	1988
Wednesday	1939
Thursday	1936

dtype: int64

```
In [29]: df['month'].value_counts()
```

```
Out[29]:
```

	count
month	
8	1654
7	1635
6	1421
9	1370
10	1333
11	1200
5	1189
12	1164
1	1069
4	875
2	808
3	799

dtype: int64

Летние месяцы неожиданно лидируют по числу аварий. Потеря бдительности с пятницы до понедельника более предсказуема.

Категории водителей по стажу. Есть ли категории, которые разительно отличаются по числу ДТП

Выгружаем другой датафрейм.

```
In [30]: df_partipiants = pd.read_csv('/datasets/Kirovskaya_oblast_participants.csv')
```

```
In [31]: df_partipiants.head()
```

Out[31]:

	role	gender	violations	health_status	years_of_driving_experience	id
0	Водитель	Мужской	['Несоответствие скорости конкретным условиям ...	Раненый, находящийся (находившийся) на амбулат...	26.0	1983180
1	Водитель	Мужской	[]	Не пострадал	34.0	2889433
2	Пассажир	Мужской	[]	Раненый, находящийся (находившийся) на амбула...	NaN	2591208
3	Пассажир	Мужской	[]	Раненый, находящийся (находившийся) на амбула...	NaN	2591208
4	Водитель	Мужской	[]	Не пострадал	27.0	2577639

Не для всех водителей заполнен столбец стажа. Это будет отдельной категорией, вдруг это водители, у которых не было прав.

```
In [32]: df_partipiants.loc[df_partipiants['role'] == "Водитель"].info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 19753 entries, 0 to 31232
Data columns (total 6 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   role                                  19753 non-null  object
1   gender                               19018 non-null  object
2   violations                           19753 non-null  object
3   health_status                       19661 non-null  object
4   years_of_driving_experience          16909 non-null  float64
5   id                                   19753 non-null  int64
dtypes: float64(1), int64(1), object(4)
memory usage: 1.1+ MB
```

Также выделим категории по десятилетиям опыта.

```
In [33]: df_partipiants['category_exp'] = pd.cut(df_partipiants['years_of_driving_experience'],
```

```
In [34]: df_partipiants['category_exp'].value_counts()
```

Out[34]:

	count
category_exp	
(0, 10]	6771
(10, 20]	5049
(20, 30]	2724
(30, 40]	1618
(40, 50]	629
(50, 60]	106
(60, 70]	12

dtype: int64

Значительно уменьшается количество аварий у водителей с 20-летним стажем.

```
In [35]: df_partipiants['category_exp_extra'] = pd.cut(df_partipiants['years_of_driving_experier
```

```
In [36]: df_partipiants['category_exp_extra'].value_counts()
```

```
Out [36]:
```

	count
category_exp_extra	
(0, 5]	3533
(5, 10]	3238
(10, 15]	2769
(15, 20]	2280
(20, 25]	1587
(25, 30]	1137
(30, 35]	921
(35, 40]	697
(40, 50]	629
(50, 60]	106
(60, 70]	12

dtype: int64

Видно уменьшение числа аварий в зависимости от опыта. Можно сказать, что тех, у кого стаж составляет 60–70 лет меньшинство.

Рекомендации заказчику: сделать столбец со стажем обязательным для заполнения. В нём 2844 пропусков — 14% от общего числа. Тех, у кого не было прав, можно фиксировать отдельно. Вероятно, что причина отсутствия прав также важна: их забрали или водитель их не получил.

Как менялось число аварий в Кировской и Московской области. Где аварий больше в относительном выражении — в пересчёте на 100 тысяч жителей (Киров входит в численность, Москва исключена)

Сначала совершим технические приготовления и добавим необходимые дополнительные столбцы.

```
In [37]: df['year'] = df['datetime'].dt.year
```

```
In [38]: df_moscow=pd.read_csv('/datasets/Moscowskaya_oblast.csv')
```

```
In [39]: df_moscow = df_moscow.rename(columns={'properties.datetime': 'datetime'})

df_moscow['datetime'] = pd.to_datetime(df_moscow['datetime'])
```

```
In [40]: df_moscow['year'] = df_moscow['datetime'].dt.year
```

Добавим вручную значения численности населения по этим двум областям. После этого разделим число аварий на численность областей и получим информацию о числе аварий на сто тысяч населения.

```
In [41]: df_dtp_on_years = df['year'].value_counts().sort_index().to_frame()
df_dtp_on_years['population'] = [13, 13, 13, 13, 13, 13, 12, 11, 11, 11]
df_dtp_on_years['dtp_on_100k'] = round(df_dtp_on_years['count'] / df_dtp_on_years['popu
df_dtp_on_years
```

Out [41]:

	count	population	dtp_on_100k
year			
2015	1532	13	117.85
2016	1468	13	112.92
2017	1452	13	111.69
2018	1612	13	124.00
2019	1568	13	120.62
2020	1498	13	115.23
2021	1524	12	127.00
2022	1386	11	126.00
2023	1486	11	135.09
2024	991	11	90.09

```
In [42]: df_moscow_dtp_on_years = df_moscow['year'].value_counts().sort_index().to_frame()
df_moscow_dtp_on_years['population'] = [72, 73, 74, 75, 76, 77, 85, 85, 86, 87]
df_moscow_dtp_on_years['dtp_on_100k'] = round(df_moscow_dtp_on_years['count'] / df_mosco
df_moscow_dtp_on_years
```

Out [42]:

	count	population	dtp_on_100k
year			
2015	6643	72	92.26
2016	5794	73	79.37
2017	5747	74	77.66
2018	5102	75	68.03
2019	5111	76	67.25
2020	4216	77	54.75
2021	3898	85	45.86
2022	3353	85	39.45
2023	3372	86	39.21
2024	2382	87	27.38

Численность населения							
1897 ^[7]	1926 ^[8]	1928 ^[9]	1937 ^[10]	1941 ^[11]	1944 ^[11]	1959 ^[12]	1970 ^[13]
1 391 990	↗4 570 836	↗4 744 200	↗8 173 289	↘5 387 400	↘3 807 400	↗5 863 003	↘5 774 529
1976 ^[14]	1979 ^[15]	1987 ^[16]	1989 ^[17]	1990 ^[18]	1991 ^[18]	1992 ^[18]	1993 ^[18]
↗6 076 000	↗6 359 385	↗6 581 000	↗6 693 623	↗6 700 180	↗6 720 248	↘6 719 121	↘6 704 332
1994 ^[18]	1995 ^[18]	1996 ^[18]	1997 ^[18]	1998 ^[18]	1999 ^[18]	2000 ^[18]	2001 ^[18]
↘6 677 307	↘6 671 785	↘6 660 192	↘6 657 710	↘6 657 149	↘6 653 499	↘6 628 173	↘6 613 469
2002 ^[19]	2003 ^[18]	2004 ^[18]	2005 ^[18]	2006 ^[18]	2007 ^[18]	2008 ^[18]	2009 ^[18]
↗6 618 538	↘6 616 879	↗6 622 017	↗6 629 703	↘6 628 107	↗6 645 672	↗6 672 773	↗6 712 582
2010 ^[20]	2011 ^[18]	2012 ^[21]	2013 ^[22]	2014 ^[23]	2015 ^[24]	2016 ^[25]	2017 ^[26]
↗7 095 120	↗7 106 215	↗7 198 686	↘7 048 084	↗7 133 620	↗7 231 068	↗7 318 647	↗7 423 470
2018 ^[27]	2019 ^[28]	2020 ^[29]	2021 ^[30]	2022 ^[2]	2023 ^[31]	2024 ^[1]	
↗7 503 385	↗7 599 647	↗7 690 863	↗8 524 665	↗8 542 257	↗8 591 736	↗8 651 260	

Численность населения								
1897 ^[4]	1915 ^[5]	1937 ^[6]	1928 ^[7]	1937 ^[6]	1939 ^[8]	1940 ^[9]	1941 ^[10]	1942 ^[11]
3 030 831	↘2 115 600	↗2 387 477	↘2 252 400	↗2 380 337	↘2 226 000	↗2 334 000	↘2 318 000	↘2 091 000
1943 ^[12]	1944 ^[13]	1945 ^[14]	1946 ^[15]	1950 ^[16]	1959 ^[17]	1968 ^[18]	1970 ^[19]	1979 ^[20]
↗2 112 000	↘2 092 000	↘1 836 000	↗1 958 000	↘1 935 000	↘1 913 600	↘1 751 600	↘1 727 348	↘1 661 546
1987 ^[21]	1989 ^[22]	1990 ^[23]	1991 ^[23]	1992 ^[23]	1993 ^[23]	1994 ^[23]	1995 ^[23]	1996 ^[23]
↗1 675 000	↗1 692 655	↘1 651 726	↘1 648 962	↘1 645 091	↘1 641 886	↘1 632 494	↘1 623 370	↘1 609 480
1997 ^[23]	1998 ^[23]	1999 ^[23]	2000 ^[23]	2001 ^[23]	2002 ^[24]	2003 ^[23]	2004 ^[23]	2005 ^[23]
↘1 595 919	↘1 581 490	↘1 568 722	↘1 553 595	↘1 536 772	↘1 503 529	↘1 498 976	↘1 479 342	↘1 461 301
2006 ^[23]	2007 ^[23]	2008 ^[23]	2009 ^[23]	2010 ^[25]	2011 ^[23]	2012 ^[26]	2013 ^[27]	2014 ^[28]
↘1 442 935	↘1 426 917	↘1 413 257	↘1 401 201	↘1 341 312	↘1 338 758	↘1 327 915	↘1 319 076	↘1 310 929
2015 ^[29]	2016 ^[30]	2017 ^[31]	2018 ^[32]	2019 ^[33]	2020 ^[34]	2021 ^[35]	2022 ^[2]	2023 ^[36]
↘1 304 348	↘1 297 474	↘1 291 684	↘1 283 238	↘1 272 109	↘1 262 402	↘1 153 680	↘1 149 176	↘1 138 112
2024 ^[1]								
↘1 129 935								

Основные выводы:

В проекте были проанализированы данные о ДТП в Кировской и Московской областях. Данные в целом пригодны для анализа: явные дубликаты не обнаружены, идентификаторы ДТП уникальны, поэтому таблицы можно корректно связывать между собой.

При этом в датасетах есть пропуски в отдельных столбцах. Наиболее важные из них связаны с адресом, координатами, схемой ДТП и водительским стажем. Такие пропуски могут ограничивать анализ.

Анализ типов данных показал, что часть столбцов можно привести к более подходящим форматам.

По дням недели больше всего ДТП происходит ближе к выходным: в пятницу, субботу и воскресенье. Вероятно, это связано с увеличением транспортной активности и поездками в выходные дни.

По месяцам заметна сезонность: количество ДТП увеличивается в тёплый период года, особенно летом. Это может быть связано с ростом числа поездок, отпускным сезоном и большей активностью участников дорожного движения.

Анализ водительского стажа показал, что чаще всего в ДТП участвуют водители со стажем до 10 лет. С увеличением стажа количество таких участников снижается.

В региональном сравнении абсолютное число ДТП выше в Московской области. При этом в пересчёте на 100 тысяч жителей относительный уровень аварийности может быть выше в Кировской области.

Рекомендации заказчику

Рекомендуется улучшить полноту заполнения ключевых полей: координат, адреса, схемы ДТП и водительского стажа. Эти данные важны для анализа аварийно-опасных участков, причин происшествий и характеристик участников ДТП.

Для поля `years_of_driving_experience` стоит добавить причины отсутствия данных: стаж неизвестен, данные не внесены, участник не имел прав, водитель скрылся или информация отсутствует в источнике.

В дальнейшем можно расширить анализ на другие регионы и дополнительно учитывать тип дороги, время суток, погодные условия, плотность населения и состояние дорожной инфраструктуры.